Priority Non-Preemptive

Program:

```
#include <stdio.h>

int main() {
  printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

  int n, curr_time = 0, pid[10], at[10], bt[10], p[10], ct[10], tat[10], wt[10],
        st[10], rt[10], i, j, completed = 0, done[10] = {0}, choice;
  double tat_sum = 0, wt_sum = 0;

  printf("Enter no. of processes: ");
  scanf("%d", &n);

  for (i = 0; i < n; i++) {
    pid[i] = i + 1;

    printf("Enter arrival time for process[%d]: ", i + 1);
    scanf("%d", &at[i]);

    printf("Enter burst time for process[%d]: ", i + 1);
    scanf("%d", &bt[i]);

    printf("Enter priority for process[%d]: ", i + 1);
    scanf("%d", &p[i]);
  }

  printf("\nType:");
  printf("1. Lower the number higher the priority\n");
  printf("2. Higher the number higher the priority\n");
  scanf("%d", &choice);

  while (completed != n) {

    int min = 99999999,  // to set with the lowest priority
        max = -99999999, // to set with the highest priority
        s = -1; // index of the process with the lowest/highest priority

    for (i = 0; i < n; i++) {
      // 1. AT is now or before current time
      // 2. priority is lesser than min
      // 3. process is not completed
      if (choice == 1) {
        if (at[i] <= curr_time && p[i] < min && !done[i]) {
          min = p[i];
          s = i;
        }
      }
```

```
    if (choice == 2) {
      if (at[i] <= curr_time && p[i] > max && !done[i]) {
        max = p[i];
        s = i;
      }
    }
  }

  // CPU idle time
  if (s == -1) {
    curr_time++;
    continue;
  }

  st[s] = curr_time;
  curr_time += bt[s];

  // CT
  ct[s] = curr_time;

  // TAT
  tat[s] = ct[s] - at[s];
  tat_sum += tat[s];

  // WT
  wt[s] = tat[s] - bt[s];
  wt_sum += wt[s];

  // RT
  rt[s] = st[s] - at[s];

  // increment the completed processes count
  completed++;
  done[s] = 1;
}

// header row
printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n\n");

// sorted by PID
for (j = 1; j < n + 1; j++) {
  for (i = 0; i < n; i++) {
    if (pid[i] == j)
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], p[i], ct[i],
             tat[i], wt[i], rt[i]);
  }
}
```

```
  printf("\nAverage TAT: %.2f\n", (tat_sum / n));
  printf("Average WT: %.2f\n", (wt_sum / n));

  return 0;
}
```

Output:

```
Name: ROHAN PODDAR
USN:1WA24CS238

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter priority for process[1]: 3
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter priority for process[2]: 1
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter priority for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6
Enter priority for process[4]: 2

Type:
1. Lower the number higher the priority
2. Higher the number higher the priority
Choice: 1

P       AT      BT      PR      CT      TAT     WT      RT

P1      0       7       3       7       7       0       0
P2      8       3       1       16      8       5       5
P3      3       4       4       20      17      13      13
P4      5       6       2       13      8       2       2

Average TAT: 10.00
Average WT: 5.00
```

Priority Preemptive

Program:

```
#include <stdio.h>

int main() {
  printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

  int n, curr_time = 0, pid[10], at[10], bt[10], remt[10], p[10], ct[10],
        tat[10], wt[10], st[10], rt[10], i, j, completed = 0, choice;
  double tat_sum = 0, wt_sum = 0;

  printf("Enter no. of processes: ");
  scanf("%d", &n);

  for (i = 0; i < n; i++) {
    pid[i] = i + 1;

    printf("Enter arrival time for process[%d]: ", i + 1);
    scanf("%d", &at[i]);

    printf("Enter burst time for process[%d]: ", i + 1);
    scanf("%d", &bt[i]);
    remt[i] = bt[i];

    printf("Enter priority for process[%d]: ", i + 1);
    scanf("%d", &p[i]);
  }

  printf("\nType:");
  printf("1. Lower the number higher the priority\n");
  printf("2. Higher the number higher the priority\n");
  scanf("%d", &choice);

  while (completed != n) {

    int min = 99999999,  // to set with the lowest priority
        max = -99999999, // to set with the highest priority
        s = -1; // index of the process with the lowest/highest priority

    for (i = 0; i < n; i++) {
      // 1. AT is now or before current time
      // 2. priority is lesser than min
      // 3. there is some time remaining
      if (choice == 1) {
        if (at[i] <= curr_time && p[i] < min && remt[i] > 0) {
          min = p[i];
          s = i;
        }
```

```
    }
    if (choice == 2) {
      if (at[i] <= curr_time && p[i] > max && remt[i] > 0) {
        max = p[i];
        s = i;
      }
    }
  }

  // CPU idle time
  if (s == -1) {
    curr_time++;
    continue;
  }

  // if the process is in the CPU for the first time
  if (remt[s] == bt[s])
    st[s] = curr_time;

  // decrement the remaining time of the process
  remt[s]--;

  // if the process has completed
  if (remt[s] == 0) {
    // CT
    ct[s] = curr_time + 1;

    // TAT
    tat[s] = ct[s] - at[s];
    tat_sum += tat[s];

    // WT
    wt[s] = tat[s] - bt[s];
    wt_sum += wt[s];

    // RT
    rt[s] = st[s] - at[s];

    // increment the completed processes count
    completed++;
  }
  curr_time++;
}

// header row
printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n\n");

// sorted by PID
```

```
  for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
      if (pid[i] == j)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], p[i], ct[i],
               tat[i], wt[i], rt[i]);
    }
  }

  printf("\nAverage TAT: %.2f\n", (tat_sum / n));
  printf("Average WT: %.2f\n", (wt_sum / n));

  return 0;
}
```

Output:

```
Name: ROHAN PODDAR
USN:1WA24CS238

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter priority for process[1]: 3
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter priority for process[2]: 1
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter priority for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6
Enter priority for process[4]: 2

Type:
1. Lower the number higher the priority
2. Higher the number higher the priority
Choice: 2

P       AT      BT      PR      CT      TAT     WT      RT

P1      0       7       3       11      11      4       0
P2      8       3       1       20      12      9       9
P3      3       4       4       7       4       0       0
P4      5       6       2       17      12      6       6

Average TAT: 9.75
Average WT: 4.75
```

Round Robin

Program:

```
#include <stdio.h>

int main() {
  printf("Name: ROHAN PODDAR\nUSN:1WA24CS238\n\n");

  int n, completed = 0, curr_time = 0, pid[10], at[10], bt[10], remt[10],
        ct[10], tat[10], wt[10], st[10], rt[10], i, j, tq, q[10], f = 0,
        r = -1, inq[10] = {0};
  double tat_sum = 0, wt_sum = 0;

  printf("Enter no. of processes: ");
  scanf("%d", &n);

  for (i = 0; i < n; i++) {
    pid[i] = i + 1;

    printf("Enter arrival time for process[%d]: ", i + 1);
    scanf("%d", &at[i]);

    printf("Enter burst time for process[%d]: ", i + 1);
    scanf("%d", &bt[i]);
    remt[i] = bt[i]; // initialising remaining time
  }

  printf("Enter time quantum: ");
  scanf("%d", &tq);

  // running a while loop for each time interval
  // until all the processes have completed
  while (completed != n) {

    // add processes that have arrived to queue
    for (i = 0; i < n; i++) {
      if (at[i] <= curr_time && remt[i] > 0 && !inq[i]) {
        r = (r + 1) % 10;
        q[r] = i;
        inq[i] = 1;
      }
    }

    // CPU idle time
    if (f > r) {
      curr_time++;
      continue;
    }
```

```
// dequeue
i = q[f];
f = (f + 1) % 10;
inq[i] = 0;

// if the process is in the CPU for the first time
if (remt[i] == bt[i])
  st[i] = curr_time;

// decrement the remaining time of the process
if (remt[i] < tq) {
  curr_time += remt[i];
  remt[i] = 0;
} else {
  curr_time += tq;
  remt[i] -= tq;
}

// add newly arrived processes during execution
for (j = 0; j < n; j++) {
  if (at[j] <= curr_time && remt[j] > 0 && !inq[j] && j != i) {
    r = (r + 1) % 10;
    q[r] = j;
    inq[j] = 1;
  }
}

// if the process has completed
if (remt[i] == 0) {
  // CT
  ct[i] = curr_time;

  // TAT
  tat[i] = ct[i] - at[i];
  tat_sum += tat[i];

  // WT
  wt[i] = tat[i] - bt[i];
  wt_sum += wt[i];

  // RT
  rt[i] = st[i] - at[i];

  // increment the completed processes count
  completed++;
} else {
  r = (r + 1) % 10;
  q[r] = i;
```

```
      inq[i] = 1;
    }
  }

  // header row
  printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");

  // sorted by PID
  for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
      if (pid[i] == j)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
               tat[i], wt[i], rt[i]);
    }
  }

  printf("\nAverage TAT: %.2f\n", (tat_sum / n));
  printf("Average WT: %.2f\n", (wt_sum / n));

  return 0;
}
```

Output:

```
Name: ROHAN PODDAR
USN:1WA24CS238

Enter no. of processes: 4
Enter arrival time for process[1]: 0
Enter burst time for process[1]: 7
Enter arrival time for process[2]: 8
Enter burst time for process[2]: 3
Enter arrival time for process[3]: 3
Enter burst time for process[3]: 4
Enter arrival time for process[4]: 5
Enter burst time for process[4]: 6
Enter time quantum: 3

P       AT      BT      CT      TAT     WT      RT

P1      0       7       17      17      10      0
P2      8       3       16      8       5       5
P3      3       4       13      10      6       0
P4      5       6       20      15      9       4

Average TAT: 12.50
Average WT: 7.50
```